THE ARCHITECT'S FIELD NOTES

BEHAVIORAL INTERVIEW STUDY GUIDE / 06 OF 07

System Design #3 — E-Commerce Checkout

THINK CLEARLY. DECIDE WITH EVIDENCE. DELIVER WITH CONFIDENCE.

Evidence-based STAR answers, senior-level judgment, and focused interview rehearsal.

PREPARED FOR SENIOR / PRINCIPAL SOLUTION ARCHITECT INTERVIEWS

READ · REFLECT · PRACTICE

06



YOUR STUDY COMPANION

Priority: High — Distributed transactions **Interview target:** Solution Architect **Format:** Deep design, with decisions, alternatives, and failure recovery

> **Core problem:** A customer purchases multiple products. We must create the order, reserve inventory, process payment, and start fulfillment without charging twice or selling stock we do not have.
>
> The challenge is not connecting four services. It is ensuring they reach a correct business outcome when one or more services fail.

NAVIGATION

Inside this guide

01	1. Understand the problem
02	2. First derive the simplest architecture
03	3. Saga orchestration versus choreography
04	4. High-level system architecture
05	5. API contracts
06	6. Data model
07	7. Successful checkout flow and why each step exists
08	8. Prevent inventory overselling
09	9. Saga failure recovery
10	10. Transactional Outbox and the dual-write problem
11	11. Handle payment webhooks
12	12. Scaling and performance
13	13. Security and compliance
14	14. Reliability and operations
15	15. Final decision summary
16	16. Concise interview walkthrough
17	Official references for further verification

CHAPTER

1. Understand the problem

Functional requirements

1. Customer checks out with multiple products.
2. System validates product availability and calculates the final price.
3. System reserves inventory to prevent overselling.
4. System authorizes payment through an external payment provider.
5. System confirms the order after the required checks succeed.
6. System starts fulfillment and sends confirmation.
7. System handles failed payments, expired reservations, duplicate requests, and partial failures.
8. Customer can check the order's current status.

Assumption: A physical-goods retailer using card payments.

Out of scope: Product search, recommendations, catalog browsing, detailed shipping optimization, and chargebacks.

Non-functional requirements

Requirement	Why it matters	Design implication
No duplicate orders or charges	Financial correctness	Idempotency
No overselling	Inventory correctness	Atomic reservation
High availability	Checkout generates revenue	Redundancy and recovery
Responsive checkout	Customer experience	Short critical path
Fault tolerance	External systems fail	Retries, Saga, compensation
Recoverable operations	Failures can occur midway	Durable workflow state
Security	Payments and personal information	Authentication, tokenization, encryption
Observability	Investigate stuck or failed orders	Correlation IDs, metrics, audit trail

Design priority: Correctness > availability during uncertainty > performance optimization.

A temporary `PROCESSING` status is preferable to telling the customer payment failed when the actual outcome is unknown.

Illustrative capacity assumptions

These are design assumptions, not measured company requirements.

Metric	Assumption
Orders per day	10 million
Average checkout rate	~116 requests/second
Peak multiplier	10×
Peak checkout rate	~1,160 requests/second
Average items per order	3

Order record and related metadata	~2 KB

At this scale, a properly designed relational database can be a reasonable starting point. Do not automatically introduce database sharding or dozens of microservices.

The harder problem is **transaction coordination**, not just requests per second.

CHAPTER

2. First derive the simplest architecture

Imagine everything runs inside one application and one relational database:

```
Customer
   |
   v
Checkout Application
   |
   +--> Validate cart
   |
   +--> Check inventory
   |
   +--> Save order
   |
   +--> Process payment
   |
   +--> Send confirmation
   |
   v
Database
```

Now consider:

1. Save the order successfully.
2. Reduce inventory successfully.
3. Call the external payment processor.
4. Payment succeeds.
5. The application crashes before saving payment success.

We now have inconsistent information:

- Order database may say payment is pending.
- Inventory may already be reduced.
- Payment provider has authorized or charged the customer.
- Customer does not know the outcome.

A single SQL transaction does not solve this. It can atomically commit changes within its transaction boundary but generally cannot include an external payment provider.

Decision: use a Saga

A **Saga** coordinates a distributed business workflow as a series of local transactions, using recovery or compensating actions when needed.

```
Create Order
     |
     v
Reserve Inventory
     |
     v
Authorize Payment
     |
     v
Confirm Order
     |
     v
Start Fulfillment
```

On a declined payment:

```
Payment Declined
      |
      v
Release Inventory
      |
      v
Cancel Order
```

Important: Compensation is not an automatic rollback. Releasing inventory is a new business action. Voiding an authorization or issuing a refund may require a separate external financial operation.

CHAPTER

3. Saga orchestration versus choreography

Choreography

```
Order
  |
Event
  v
Inventory
  |
Event
  v
Payment
```

Services react to events and trigger subsequent steps.

Orchestration

```
      Checkout Saga
       /    |    \
      v     v     v
   Order Inventory Payment
```

A central durable workflow coordinates participants.

Aspect	Choreography	Orchestration
Control	Distributed	Central coordinator
Workflow visibility	Harder	Easier
Service coupling	Event contracts	Orchestrator contracts
Complex branching	Harder to follow	Easier to manage
Recovery	Spread across services	Explicit workflow decisions

Choice: Saga orchestration

Why: Checkout has a defined sequence, important compensation steps, and multiple failure cases. The coordinator tracks which steps succeeded, which failed, whether payment outcome is known, and what recovery is safe.

Critical requirement: Persist workflow state. An in-memory-only orchestrator loses progress on crashes.

When choreography can work: Simpler event-driven processes with few participants and uncomplicated recovery.

CHAPTER

4. High-level system architecture



```
                        v
                Event Relay / CDC
                        |
                        v
                 Message Broker
                        |
              +---------+----------+
              |                    |
              v                    v
      Fulfillment Service   Notification Service
```

PSP = Payment Service Provider; CDC = Change Data Capture. Each publishing service owns an outbox in its own transactional database.

Component	Responsibility	Why separate it logically?
API Gateway	Routing, authentication, limits	Common entry controls
Checkout API	Validate request, start checkout	Customer-facing interface
Pricing Service	Final price, tax, discounts	Do not trust browser-supplied totals
Saga Orchestrator	Manage checkout steps	Central recovery decisions
Order Service	Own order lifecycle	Clear data ownership
Inventory Service	Reserve and release items	Prevent overselling
Payment Service	Integrate with PSP	Isolate payment rules/failures
Outbox/Event Relay	Publish durable events	Avoid lost events
Message Broker	Deliver asynchronous work	Decouple downstream services
Fulfillment Service	Arrange delivery	Independent later workflow
Notification Service	Send email/SMS	Not critical-path financial logic

Architecture boundary: Separate deployable services and databases are justified by ownership, scaling, or reliability needs. Otherwise, a **modular monolith** with a durable workflow and relational database may be a better starting implementation. The correctness rules apply in either case.

CHAPTER

5. API contracts

Start checkout

```
POST /v1/checkouts
Authorization: Bearer <access_token>
Idempotency-Key: checkout-attempt-123
Content-Type: application/json
```

```
{
  "cart_id": "cart_101",
```

```
  "cart_version": 4,
  "shipping_address_id": "addr_50",
  "payment_method_token": "pm_token_123"
}
```

The server derives the authenticated customer and calculates the final amount; it must not trust browser prices.

Asynchronous response

```
{
  "checkout_id": "chk_123",
  "order_id": "ord_456",
  "status": "PROCESSING",
  "amount_minor": 12500,
  "currency": "USD"
}
```

Return **HTTP 202 Accepted** when processing continues asynchronously.

```
GET /v1/checkouts/chk_123
```

Example states:

```
PROCESSING
REQUIRES_ACTION
CONFIRMED
FAILED
CANCELLED
```

`REQUIRES_ACTION` may cover additional payment authentication, such as 3-D Secure.

Why idempotency?

```
Customer clicks Pay
       |
       v
Server processes request
       |
       v
Network loses response
       |
       v
Customer retries
```

Without idempotency, retries could create another order.

Implementation: Store authenticated customer, idempotency key, request fingerprint, and checkout attempt ID. Enforce a unique constraint so concurrent retries do not start duplicate workflows. Reject reuse of the same key with different parameters.

Important: Checkout-level and PSP-level idempotency are separate safeguards. Use both.

CHAPTER

6. Data model

Choice: relational database for transactions

Why SQL is a sensible default:

- Orders have related items and payments.
- Inventory reservation needs conditional atomic updates.
- Payment state needs constraints and safe transitions.
- Local transactions enforce consistency within a service boundary.

A Saga coordinates across separately committed service transactions.

Table	Important fields
checkout_attempts	attempt*id, customer*id, idempotency*key, request*hash, status
orders	order*id, customer*id, total_minor, currency, status, version
order_items	order*id, product*id, quantity, unit*price*minor
inventory	sku, warehouse*id, on*hand, reserved
reservations	reservation*id, order*id, sku, quantity, expires_at, status
payments	payment*id, order*id, provider*reference, amount*minor, status
saga_instances	saga*id, order*id, current_step, status, version
outbox_events	event*id, aggregate*id, event*type, payload, publish*status
processed_events	consumer*id, event*id, processed_at

In a service-owned design, place each table in the appropriate service database, not in one shared cross-service schema.

Money: Store integer minor units (for example, 12500 cents for USD 125.00) or a properly defined decimal type, never binary floating point for financial arithmetic. Record currency and rounding rules.

Historical truth: Snapshot product prices and purchase details into the order so catalog changes do not rewrite order history.

CHAPTER

7. Successful checkout flow and why each step exists

1. **Validate checkout and calculate price.** Check identity, cart version, shipping, availability, discounts, taxes, and currency. Have the user accept the final server-calculated total. *Why first?* Avoid resource reservations and payments for an invalid cart.
2. **Persist checkout attempt and pending order.** Record the operation identity and Saga state durably. *Why?* The workflow must survive a crash before side effects.
3. **Reserve inventory.** Inventory Service checks and reserves all requested quantities atomically within its transactional boundary. *Why before payment?* Do not authorize a payment for unavailable stock.
4. **Authorize payment.** Payment Service creates/reuses a stable PSP operation. *Why authorization?* For suitable card/payment flows, funds can be held before capture under fulfillment policy.

5. **Commit inventory allocation.** Convert temporary reservations to allocated inventory idempotently. *Why?* A confirmed order must not rely on an expiring temporary hold.
6. **Confirm order.** Change order to `CONFIRMED`, writing `OrderConfirmed` to the outbox in the same local transaction. *Why?* The order now reflects completed prerequisites.
7. **Start fulfillment and notifications.** Relay events; consumers act independently. *Why asynchronously?* Email or shipping-service problems must not invalidate a confirmed purchase.

Authorization versus capture

Operation	Meaning
Authorization	Request approval to hold available funds
Capture	Request collection of previously authorized funds
Void/cancel authorization	Release an authorization no longer needed
Refund	Return funds after a completed payment

For this example, assume capture happens at an appropriate fulfillment/shipping stage. Real provider rules, payment methods, authorization expiry, and merchant policy determine exact timing.

`CONFIRMED` is not the same as settled. Here it means inventory allocated and payment authorized.

CHAPTER

8. Prevent inventory overselling

Two customers attempt to buy the last item:

```
Inventory = 1

Customer A checks stock --> 1 available
Customer B checks stock --> 1 available

Customer A reserves 1
Customer B reserves 1

Incorrect result: 2 orders for 1 item
```

Wrong pattern: read then update

```
SELECT available FROM inventory WHERE sku = 'ABC';
-- Application sees 1 available.
UPDATE inventory
SET reserved = reserved + 1
WHERE sku = 'ABC';
```

Concurrent readers can both observe the same stock.

Better: conditional atomic update

```
UPDATE inventory
SET reserved = reserved + 1
WHERE sku = 'ABC'
  AND on_hand - reserved >= 1;
```

If the affected row count is zero, the reservation fails. The condition and update occur atomically according to the database's concurrency semantics.

For an order with multiple SKUs, try to reserve all relevant rows in **one local Inventory Service transaction**, with a consistent lock order to reduce deadlocks. If stock spans independent databases, separate reservations require compensation for partial success.

Why not Redis as the source of truth?

Redis is useful for fast **approximate product availability**. Cache expiry, replication delays, and failover can make it unsuitable as the final authority for scarce inventory.

Choice: Final stock allocation uses durable, transactional inventory records. Cache read-only availability displays.

Reservation lifecycle

```
AVAILABLE
    |
    v
 RESERVED
    |
    +------> RELEASED
    |        payment failed / expired
    |
    v
 COMMITTED
```

An expiry worker releases reservations only after an atomic state check. If payment authorization arrives after stock expiry, attempt safe compensation (for example, void authorization) rather than confirming nonexistent inventory.

CHAPTER

9. Saga failure recovery

Failure	What can go wrong?	Correct recovery
Inventory unavailable	Charge for unavailable goods	Stop before payment; fail order
Partial reservation	Some stock held; other items unavailable	Release completed reservations
Payment declined	Stock remains held	Release stock and cancel order
Payment timeout	Payment may have succeeded	Mark unknown; query/reconcile PSP
Payment authorized, order DB unavailable	Order unconfirmed though funds held	Retry confirmation from durable Saga state
Inventory expires during payment	Late authorization without stock	Void authorization and cancel safely
Notification fails	Confirmation email not delivered	Retry; order stays confirmed

Orchestrator crashes	Workflow stops midway	Resume from persisted state
Event delivered twice	Duplicate downstream action	Idempotent consumers
Webhook arrives late	Local state is outdated	Reconcile with provider state

Critical scenario: payment authorized, order update fails

```
Inventory reserved
       |
       v
Payment authorized
       |
       v
Order DB unavailable
       |
       v
Payment and order outcomes diverge
```

Do not tell the customer payment failed just because the order database update failed.

1. Recover the PSP state and provider transaction reference.
2. Keep a recoverable Saga state.
3. Retry order confirmation when safe.
4. Verify inventory allocation remains valid.
5. If completion is impossible, void the authorization or refund as appropriate.
6. Reconcile unresolved records.

Key distinction: Retry recoverable technical failures. Compensate if the business workflow cannot complete.

Why blindly retrying a PSP call is dangerous

A timeout does not prove the initial PSP operation failed. Another fresh request may create an additional authorization. Reuse a stable provider idempotency key or operation reference and determine the existing result first.

Never treat a payment timeout as proof of failure.

CHAPTER

10. Transactional Outbox and the dual-write problem

Unsafe sequence:

```
1. UPDATE orders SET status = 'CONFIRMED'
2. Publish OrderConfirmed to message broker
```

If the DB update succeeds and publishing fails, the order is confirmed but fulfillment never receives the event. Publishing first can create the reverse inconsistency.

Choice: Transactional Outbox

```
          Order Service
               |
               v
       ONE DB TRANSACTION
       +------------------+
       | Update Order     |
       | Insert Outbox    |
       +------------------+
               |
             COMMIT
               |
               v
           Outbox Relay
               |
               v
          Message Broker
               |
               v
        Fulfillment Service
```

The order update and outbox insertion commit together. A relay publishes committed outbox records and retries if necessary.

Does outbox guarantee exactly-once delivery?

No. For example, the relay might publish successfully and crash before recording that fact. On restart, it publishes again.

```
Reliable Outbox
      +
At-least-once Delivery
      +
Idempotent Consumer
      =
Reliable Business Processing
```

The consumer stores an event ID or business operation ID under a uniqueness constraint. Its deduplication record and business update must commit atomically.

Kafka versus queue

Requirement	Better initial fit
Independent jobs distributed to workers	Durable work queue
Replay event history	Kafka-style stream
Multiple independent consumers	Pub/sub or stream
Per-aggregate ordering	Partition/key by aggregate ID
Simple notification delivery	Queue
Large-scale analytical streaming	Kafka-style stream

Brokers do not automatically provide global ordering or exactly-once business effects.

Choice: Start with a durable queue or event bus for checkout. Use a log/stream platform where replay, retention, and high-throughput event processing are real requirements.

CHAPTER

11. Handle payment webhooks

PSP webhooks may arrive more than once, out of order, or after local timeouts. Do not assume the last received event is the current truth.

```
Payment Provider
       |
       v
Webhook Endpoint
       |
       v
Verify Signature
       |
       v
Durable Event Inbox
       |
       v
Queue / Worker
       |
       v
Check Provider State
       |
       v
Update Payment State
       |
       v
Resume Saga
```

Why: Separate reliable acceptance of a notification from slower processing. Verify signatures, persist the event, deduplicate, and apply valid payment state transitions. If there is uncertainty, reconcile against the PSP's authoritative state.

For extra cardholder authentication, the workflow may enter `REQUIRES_ACTION` and resume after the customer completes the step.

CHAPTER

12. Scaling and performance

Concern	Design decision	Why
Growing API traffic	Stateless replicas behind a load balancer	Horizontal scaling
Repeated catalog reads	Redis/cache/CDN	Reduce read demand
Hot products	Atomic inventory updates; partition appropriately	Stock correctness and contention control
Large order-history reads	Read replicas or projections	Separate reads from writes
Increasing event volume	Partitioned broker and worker consumers	Parallelism
Slow PSP	Timeouts, bounded retries, limited concurrency	Avoid resource exhaustion
Large historical tables	Archiving/partitioning when needed	Maintenance and query efficiency

Should we shard immediately?

No. Measure database CPU, storage, contention, latency, and throughput first.

```
Single SQL Primary
       |
       v
Primary + Read Replicas
       |
       v
Partitioned Large Tables
       |
       v
Sharding by suitable ownership key
```

Sharding creates cross-shard transaction and operational complexity. Inventory and orders may need different partition keys: inventory by SKU and warehouse, orders by customer or order ID.

Multi-region correctness

Two active regions independently selling the last shared unit can oversell.

A safer first design gives each inventory allocation domain a single authoritative write owner, with well-defined regional failover or separate regional stock ownership. Active-active writable inventory requires explicit coordination/conflict strategy.

CHAPTER

13. Security and compliance

- **Authentication:** Validate account or guest checkout session.
- **Authorization:** Users can access only their own carts, orders, and payment operations.
- **Server validation:** Never trust browser-supplied amounts or discounts.
- **Tokenization:** Prefer hosted provider payment collection/SDKs that avoid direct card handling.
- **TLS:** Protect external and sensitive internal traffic.
- **Secrets management:** Store PSP API keys securely, rotate, least privilege.
- **Webhook verification:** Verify signatures and handle replay.
- **Data protection:** Encrypt sensitive data and remove credentials from logs.
- **PCI DSS:** Determine actual cardholder-data scope and relevant controls.

Using a third-party payment provider does **not** automatically eliminate a merchant's PCI DSS responsibilities.

For APIs, enforce object-level authorization: modifying `order_id` must never expose another customer's purchase.

CHAPTER

14. Reliability and operations

Important metrics

Metric	What it reveals

Checkout completion rate	Business success
Checkout p95/p99 latency	Customer experience
Payment authorization failures	PSP/payment problems
Unknown payment outcomes	Reconciliation risk
Inventory reservation failures	Stock contention
Stuck Saga count and age	Workflow recovery problems
Outbox publication lag	Event delivery health
Consumer retries/DLQ count	Downstream failures
Authorization/capture mismatches	Financial reconciliation risk

Resilience measures

- Deploy stateless components across availability zones where supported.
- Use durable state, health checks, bounded retries, circuit breakers, and controlled concurrency.
- **Circuit breaker:** Temporarily stop calls to a failing dependency to limit cascading failures; do not falsely classify uncertain payments as failed.
- Propagate correlation IDs through APIs, Saga actions, PSP references, and events.
- Reconcile local payment/order state with PSP records periodically, not only after errors.
- Test PSP timeouts, duplicate webhooks, broker redelivery, DB failures, and reservation expiry.

CHAPTER

15. Final decision summary

Decision	Chosen approach	Main reason
Transaction coordination	Orchestrated Saga	Explicit workflow and compensation ownership
Transactional data	Relational SQL	Local ACID and constraints
Inventory	Atomic reservations	Avoid overselling
Payments	Stable PSP operation and idempotency	Safe retries
Capture	Separate from authorization when appropriate	Align collection with fulfillment
Communication	Sync for needed decisions; async downstream	Performance and resilience
Event publication	Transactional Outbox	Avoid lost events
Event delivery	At-least-once + idempotent consumers	Safe redelivery
Failure recovery	Bounded retries + reconciliation	Manage uncertain outcomes
Caching	Read optimization, not inventory authority	Preserve correctness
Deployment	Start simple; scale measured bottlenecks	Avoid unnecessary complexity

CHAPTER

16. Concise interview walkthrough

> We start with a checkout API that validates the cart, calculates the final server-side price, and creates a durable checkout attempt identified by an idempotency key.
>
> Because orders, inventory, and payments are separate transactional boundaries, we coordinate them through an orchestrated Saga. The Saga creates a pending order, reserves stock atomically, and requests payment authorization from an external provider. Once the stock is committed and the payment is authorized, the order is confirmed.
>
> We use a Transactional Outbox to publish the confirmation event reliably. Fulfillment and notifications process it asynchronously. Consumers are idempotent because events can be delivered more than once.
>
> For recovery, we distinguish technical failures from business failures. Declines release inventory. Timeouts with uncertain payment outcomes trigger reconciliation rather than blind retries. We track durable workflow progress, scale stateless APIs, protect authoritative stock updates, and monitor stuck workflows and payment mismatches.

Five concepts to remember

1. **Saga:** Coordinates a business workflow across independent transactions.
2. **Compensation:** Performs a new action to reverse a previously completed business step.
3. **Idempotency:** Prevents duplicate effects on retries of the same logical operation.
4. **Transactional Outbox:** Reliably connects a database change to event publication.
5. **Atomic reservation:** Prevents multiple buyers from using the same stock.

Central lesson: Distributed checkout cannot assume all services succeed together. Design explicitly for partial success, retries, uncertain external outcomes, compensation, and eventual business completion.

CHAPTER

Official references for further verification

- AWS Prescriptive Guidance — Saga orchestration pattern: https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/saga-orchestration.html
- AWS Prescriptive Guidance — Transactional Outbox pattern: https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html
- Stripe — Idempotent requests: https://docs.stripe.com/api/idempotent_requests
- Stripe — Webhook handling: https://docs.stripe.com/webhooks
- Stripe — Separate authorization and capture: https://docs.stripe.com/payments/place-a-hold-on-a-payment-method
- AWS SQS — At-least-once delivery: https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/standard-queues-at-least-once-delivery.html
- PCI Security Standards Council: https://www.pcisecuritystandards.org/
- OWASP API Security Top 10: https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/